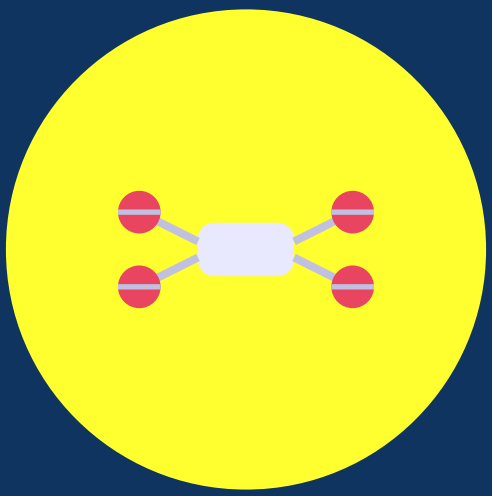

FLY-IN

Rapport d'Analyse du Projet

42 Curriculum | Python | Pathfinding | v1.2

Généré le 15 avril 2026

1. Vue d'Ensemble du Projet

Fly-in est un projet de la formation 42 demandant de concevoir un **systeme de routage de drones** dans un reseau de zones connectees. L'objectif est de faire transiter une flotte de drones d'une zone de depart vers une zone d'arrivee en un **minimum de tours de simulation**, tout en respectant un ensemble de contraintes strictes de capacite et de type de zone.

Python 3.10+	OOP strict	Type-safe	Graphes	Pathfinding
Langage obligatoire	100% oriente objet	mypy + flake8	Sans networkx	Algo optimise

Objectif Principal

> Acheminer **tous les drones** du hub de depart au hub d'arrivee en un **minimum de tours de simulation**. Le score est directement le nombre de tours total — moins il y en a, mieux c'est.

Contraintes Cles

- **Bibliotheques graphes interdites** : pas de networkx, graphlib, etc.
- **100% Oriente Objet** : toutes les structures doivent etre des classes.
- **Type-safety obligatoire** : mypy sans erreur, flake8 conforme.
- **Gestion des exceptions** : aucun crash tolere, try/except partout.
- **Docstrings PEP 257** : toutes les fonctions et classes documentees.
- **Makefile** : regles install, run, debug, clean, lint obligatoires.
- **README.md** en anglais avec description algo + visuels.

Types de Zones

Type	Cout (tours)	Comportement
`normal`	1	Zone standard, deplacement classique
`priority`	1 (prefere)	Prefere dans le pathfinding, meme cout
`restricted`	2	Zone sensible, le drone passe 2 tours en transit
`blocked`	Inaccessible	Interdit — aucun chemin ne peut le traverser

Chaque connexion peut aussi avoir un attribut **max_link_capacity** (defaut 1) limitant le nombre de drones pouvant la traverser simultanement.

2. Format des Fichiers d'Entree

Le programme lit un fichier texte decrivant le graphe. Voici la grammaire exacte :

```
nb_drones: 5 start_hub: hub 0 0 [color=green] end_hub: goal 10 10 [color=yellow] hub:
zoneA 3 4 [zone=restricted color=red max_drones=2] hub: zoneB 6 2 [zone=normal
color=blue] hub: zoneC 5 5 [zone=blocked color=gray] connection: hub-zoneA connection:
zoneA-zoneB [max_link_capacity=2] connection: zoneB-goal # Ceci est un commentaire
```

Regles du parser

- Premiere ligne = **nb_drones** (entier positif).
- Exactement **un start_hub** et **un end_hub**.
- Chaque zone a un nom unique (pas de tirets ni d'espaces dans les noms).
- Les connexions ne referencent que des zones deja definies.
- Pas de connexion dupliquee (a-b et b-a sont equivalents).
- Les types de zone invalides levant une erreur de parsing.
- Les capacites (max_drones, max_link_capacity) doivent etre des entiers > 0.
- Toute erreur arrete le programme avec un message clair (ligne + cause).
- Les commentaires commencent par '#' et sont ignores.

3. Mecaniques de Simulation

Regles de deplacement (par tour)

- Un drone peut **se deplacer** vers une zone adjacente si la capacite le permet.
- Un drone vers une zone **restricted** occupe la connexion pendant 2 tours et **doit** atteindre la destination au tour suivant.
- Un drone peut **attendre** sur place si aucun mouvement n'est possible.
- Les drones qui quittent une zone **liberent leur place** dans le meme tour.
- La zone de depart autorise plusieurs drones simultanement (exception).
- La zone d'arrivee est illimitee — tout drone qui y arrive est livre.

Format de Sortie

Chaque ligne represente **un tour**. Format : **D<ID>-<zone>** space-separe. Exemple :

```
D1-roof1 D2-corridorA
D1-roof2 D2-tunnelB
D1-goal D2-goal
```

Criteres de Scoring

Score principal = nombre total de tours. Metriques secondaires (optionnelles mais recommandees) :

- Nombre de drones deplaces par tour.
- Nombre moyen de tours par drone.
- Cout total de chemin (somme des couts ponderes).
- Qualite de la representation visuelle.

4. Architecture Recommandee

Le projet doit etre entierement en Programmation Orientee Objet. Voici l'architecture de classes suggeree :

Classe	Responsabilite
`Zone`	Represente un hub : nom, coordonnees, type, couleur, capacite max, drones presents.
`Connection`	Lien bidirectionnel entre deux zones : capacite max, drones en transit.
`Graph`	Contient les zones et connexions. Methodes : add_zone, add_connection, get_neighbors, find_paths.
`Drone`	Etat courant : position, chemin prevu, tour d'arrivee si en transit vers restricted.
`Parser`	Lit et valide le fichier d'entree. Retourne un Graph + nb_drones.
`PathFinder`	Algorithme(s) de recherche de chemin : A*, BFS pondere, recherche de chemins disjoints.
`Scheduler`	Planifie les mouvements tour par tour. Resout les conflits de capacite.
`Simulator`	Orchestre Parser -> PathFinder -> Scheduler -> affichage.
`Renderer`	Affichage colore terminal (colorama/rich) et/ou interface graphique (tkinter/pygame).

5. Algorithme de Pathfinding

L'algorithme cle du projet est le **routage multi-agents** sur graphe pondere avec contraintes de capacite. Voici l'approche recommandee :

Phase 1 — Recherche de chemins

Utiliser **A* (A-star)** ou **Dijkstra** pour trouver les chemins les plus courts entre start et end, en tenant compte du cout par type de zone. Chercher plusieurs chemins **disjoints** (Edge-Disjoint Paths) pour distribuer les drones en parallele.

```
# Heuristique A* (distance Manhattan) def heuristic(zone_a: Zone, zone_b: Zone) ->
float: return abs(zone_a.x - zone_b.x) + abs(zone_a.y - zone_b.y)
```

Phase 2 — Planification temporelle

Une fois les chemins calcules, assigner les drones aux chemins et planifier les departs avec un **decalage** (offset) pour eviter les conflits de capacite. Le scheduler simule tour par tour :

1. Pour chaque tour, evaluer les mouvements possibles de chaque drone.
2. Verifier les capacites zones (max_drones) et connexions (max_link_capacity).
3. Donner la priorite aux drones les plus avances sur leur chemin.
4. Enregistrer les deplacements et mettre a jour l'etat global.
5. Repeter jusqu'a ce que tous les drones soient arrives.

Algorithmes a connaitre

Algorithme	Usage	Complexite
`A* (A-star)`	Chemin optimal avec heuristique	`O((V+E) log V)`
`Dijkstra`	Chemin le plus court pondere	`O((V+E) log V)`
`BFS`	Chemin le plus court (cout uniforme)	`O(V+E)`
`Ford-Fulkerson`	Flot maximum / chemins disjoints	`O(V * E^2)`

6. Plan d'Implementation Etape par Etape

1 Mise en place du projet

- Creer le repo Git avec .gitignore Python.
- Configurer le venv : python -m venv .venv
- Installer les deps : colorama (ou rich), pytest.
- Creer le Makefile avec les regles obligatoires.
- Initialiser la structure de fichiers (voir architecture).

2 Modelisation du graphe (OOP)

- Classe ZoneType (enum: NORMAL, RESTRICTED, PRIORITY, BLOCKED).
- Classe Zone : nom, x, y, type, couleur, max_drones, occupants.
- Classe Connection : zone1, zone2, max_link_capacity, drones_transit.
- Classe Graph : dict zones, list connections, methodes BFS/A*.
- Ajouter type hints partout + docstrings PEP 257.

3 Parser robuste

- Parser ligne par ligne, skip commentaires (#).
- Valider nb_drones (1ere ligne).
- Parser start_hub, end_hub, hub avec metadata optionnelle.
- Parser les connexions, verifier que les zones existent.
- Lever des ParseError claires avec numero de ligne.
- Tester avec map.txt fourni + vos propres cartes edge-cases.

4 Algorithme de pathfinding

- Implémenter A* sur le Graph (sans networkx).
- Cout par zone : normal=1, restricted=2, priority=1.
- Ignorer les zones blocked.
- Chercher plusieurs chemins disjoints (variante de Yen's K-paths).
- Cacher les chemins calcules pour eviter les recalculs.

5 Scheduler / Simulateur

- Assigner les drones aux chemins (distribution equitable).
- Calculer les decalages de depart pour eviter les conflits.
- Simuler tour par tour : verifier capacites, deplacer, logger.
- Gerer le cas restricted (2 tours, occupation connexion).
- Detecter et resoudre les deadlocks (attente strategique).

6 Representation visuelle

- Terminal : colorama/rich pour afficher drones par couleur de zone.
- Afficher l'etat de chaque zone a chaque tour.
- Option graphique : tkinter ou pygame pour vue reseau.
- Inclure metriques secondaires dans la sortie.

7 Tests et validation

- Tests unitaires pytest : parser, graph, pathfinder.
- Tester toutes les maps fournies, verifier les benchmarks.
- Lancer mypy . --strict et corriger les erreurs.
- Lancer flake8 . et corriger le style.
- Creer vos propres cartes pour les edge-cases.

8 README et finalisation

- README.md en anglais avec toutes les sections obligatoires.
- Documenter les choix algorithmiques et les visuels.
- Verifier que tous les fichiers sont a la racine du repo.
- Relecture complete : OOP, type hints, docstrings, Makefile.

7. Benchmarks de Performance

Le sujet fourni des cartes de test avec des objectifs de performance precis. Votre algorithme sera evalue sur ces metriques :

Carte	Drones	Cible (tours)	Difficulte
Linear path	2	<= 6	Facile
Simple fork	3	<= 6	Facile
Basic capacity	4	<= 8	Facile
Dead end trap	5	<= 15	Moyen
Circular loop	6	<= 20	Moyen
Priority puzzle	4	<= 12	Moyen
Maze nightmare	8	<= 45	Difficile
Capacity hell	12	<= 60	Difficile
Ultimate challenge	15	<= 35	Difficile
Impossible Dream	25	< 45	BONUS

Bonus : (1) Atteindre **exactement** les cibles de reference sur toutes les cartes fournies. (2) Resoudre la carte **Impossible Dream** (25 drones) en moins de 45 tours. Les bonus ne sont evalues que si la partie obligatoire est complete.

8. Technologies et Outils

Outil / Lib	Role	Obligatoire ?
`Python 3.10+`	Langage principal	OUI
`mypy`	Verification statique des types	OUI
`flake8`	Linting PEP8	OUI
`colorama / rich`	Couleurs terminal	Recommande
`pytest / unittest`	Tests unitaires	Non soumis
`tkinter / pygame`	Interface graphique	Optionnel
`heapq (stdlib)`	File de priorite pour A*/Dijkstra	Recommande
`collections (stdlib)`	deque pour BFS, defaultdict	Recommande

`dataclasses (stdlib)`	Classes de donnees typees	**Recommande**
`enum (stdlib)`	ZoneType, DroneState	**Recommande**

9. README & Soumission

Sections obligatoires du README.md

- ***This project has been created as part of the 42 curriculum by <login>.*** — Premiere ligne, italique.
- **Description** — But du projet + apercu general.
- **Instructions** — Installation, compilation, execution.
- **Resources** — Liens documentation + description de l'usage de l'IA.
- **Algorithm Choices** — Description detaillee de votre algorithme.
- **Visual Representation** — Documentation des fonctionnalites visuelles.

Checklist de Soumission

REQUIS	[x] Tous les fichiers a la racine du repo Git.
REQUIS	[x] Makefile avec regles : install, run, debug, clean, lint.
REQUIS	[x] mypy . passe sans erreur.
REQUIS	[x] flake8 . passe sans erreur.
REQUIS	[x] Docstrings sur toutes les fonctions/classes.
REQUIS	[x] Type hints sur tous les parametres et retours.
REQUIS	[x] README.md complet en anglais.
CONSEILLE	[ ] Tests unitaires (non soumis, mais prepares).
REQUIS	[x] Representation visuelle (terminal colore minimum).
REQUIS	[x] Gestion des erreurs de parsing avec messages clairs.
REQUIS	[x] Performance benchmarks atteints pour cartes easy/medium/hard.

Attention : Pendant l'evaluation, il peut vous etre demande d'**expliquer votre code** ou d'**ecrire quelques lignes en direct**. Assurez-vous de comprendre chaque partie de votre implementation, notamment le parser, l'algorithme A*, et le scheduler.